

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования

**«Вятский государственный университет»
(ФГБОУ ВО «ВятГУ»)**

Факультет автоматики и вычислительной техники

Кафедра электронных вычислительных машин

Допущено к защите

Руководитель проекта

_____/Караваева О.В./

«__»_____2025г.

Разработка МСР-сервера для Яндекс Трекера

Пояснительная записка курсового проекта по дисциплине

«Комплекс знаний бакалавра»

ТПЖА.090301.842 ПЗ

Разработал студент группы ИВТб-4301

_____/Бикметов И. Р./

Руководитель

_____/Караваева О.В./

Консультант

_____/Коржавина А.С./

Проект защищен с оценкой «_____» _____
(оценка) (дата)

Члены комиссии _____ / _____/
(подпись) (ФИО)
_____ / _____/
(подпись) (ФИО)
_____ / _____/
(подпись) (ФИО)

Киров 2025

Реферат

Бикметов И. Р. Разработка МСР-сервера для Яндекс Трекера.
ТПЖА.090301.842 ПЗ: Курс. проект / ВятГУ, каф. ЭВМ; рук. Караваева
О.В. - Киров, 2025. – ПЗ 52 с, 33 рис, 1 прил.

МСР, УМНЫЙ ПОМОЩНИК, LLM

Объект курсового проекта — техническое и программное обеспечение для работы с Яндекс Трекером.

Предмет курсового проекта — МСР-сервер.

Цель курсового проекта — Оптимизация рабочего процесса.

Результатом выполнения курсового проекта является МСР-сервер для работы с Яндекс Трекером.

Содержание

Введение	5
1 Техническое задание	6
1.1 Пользовательские истории	6
1.2 Пользовательские сценарии	7
1.3 Основания для разработки	9
1.4 Назначение разработки	10
1.4.1 Функциональные требования	10
1.4.2 Эксплуатационное назначение	11
1.5 Требования к МСР-серверу	11
1.5.1 Требования к составу выполняемых функций	12
1.5.2 Требования к организации входных и выходных данных . .	15
1.5.3 Требования к временным характеристикам	16
1.5.4 Требования к надежности	16
1.5.5 Требования к обеспечению надежного функционирования системы	16
1.5.6 Время восстановления после отказа	17
1.5.7 Условия эксплуатации	17
1.5.8 Климатические условия эксплуатации	18
1.5.9 Требования к видам обслуживания	18
1.5.10 Требования к численности и квалификации персонала . . .	19
1.5.11 Требование к маркировке и упаковке	19
1.5.12 Требования к транспортированию и хранению	19
1.5.13 Специальные требования	19
1.6 Требования к программной документации	19
1.7 Техничко-экономические показатели	20
1.8 Стадии и этапы разработки	20

Подп. и дата	Инв. № дубл.	Взам. инв. №	1.5.5	Требования к обеспечению надежного функционирования системы	16							
			1.5.6	Время восстановления после отказа	17							
			1.5.7	Условия эксплуатации	17							
			1.5.8	Климатические условия эксплуатации	18							
			1.5.9	Требования к видам обслуживания	18							
			1.5.10	Требования к численности и квалификации персонала . . .	19							
			1.5.11	Требование к маркировке и упаковке	19							
			1.5.12	Требования к транспортированию и хранению	19							
			1.5.13	Специальные требования	19							
			1.6	Требования к программной документации	19							
			1.7	Технико-экономические показатели	20							
			1.8	Стадии и этапы разработки	20							
			Подп. и дата					ТПЖА.090301.842 ПЗ				
Инв. № подл.			Изм	Лист	№ докум.	Подп.	Дата	Разработка МСР-сервера для Яндекс Трекера		Лит.	Лист	Листов
	Разраб.	Бикметов									3	40
	Пров.	Коржавина										
	Н. контр.											
	Утв.											
Кафедра ЭВМ ИВТ6-4301-04-01												

1.9	Порядок контроля и приемки	21
1.10	Заключение	21
2	Анализ предметной области	22
2.1	Общая характеристика системы	22
2.2	Анализ существующих проблем	22
2.3	Анализ требований по решению существующих проблем	24
2.4	Обзор существующих решений	26
2.5	Сравнительный анализ решений	27
2.6	Выбор архитектуры	28
2.7	Заключение	32
3	Выбор технологий и инструментов для реализации проекта	33
3.1	Анализ и обоснование выбора языка программирования	33
3.2	Выбор библиотек	35
3.3	Выбор системы контроля версий	36
3.4	Заключение	39
4	Проектирование	40

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист
										4
Изм.	Лист	№ докум.	Подп.	Дата						

Введение

В современном мире разработка программного обеспечения становится все более сложной и требует интеграции различных инструментов и сервисов. Одной из ключевых задач является обеспечение эффективного взаимодействия между различными системами управления проектами и интеллектуальными агентами на основе Large Language Models (LLM). Яндекс Трекер представляет собой мощный инструмент для управления задачами и проектами, но для его полноценной интеграции с LLM-агентами необходимы специализированные решения.

MCP (Model Context Protocol) — это протокол, разработанный для предоставления информации о проекте различным инструментам и ассистентам. MCP-серверы обеспечивают доступ к модели проекта, позволяя интеллектуальным агентам понимать структуру и содержание проекта, что значительно упрощает автоматизацию и интеграцию.

Целью данной работы является разработка MCP-сервера для Яндекс Трекера, который будет обеспечивать интеграцию между системой управления задачами и LLM-агентами. Это позволит интеллектуальным агентам получать актуальную информацию о задачах, проектах и других сущностях Яндекс Трекера, улучшая их способность к автоматизации и принятию решений.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата						
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.842 ПЗ					Лист
										5

1 Техническое задание

Техническое задание является основным документом, который описывает цели, задачи и требования к разработке проекта. В данной версии ТЗ сформулированы ключевые аспекты, необходимые для успешного выполнения проекта, учитывая потребности всех заинтересованных сторон.

1.1 Пользовательские истории

а) Руководитель проекта:

- Как руководитель проекта, я хочу иметь возможность отслеживать прогресс выполнения задач в Яндекс Трекере через LLM-ассистента, чтобы оперативно принимать управленческие решения и управлять ресурсами команды;

- Как руководитель проекта, я хочу, чтобы LLM-ассистент мог анализировать текущую загрузку команды и предлагать оптимизацию распределения задач, чтобы повысить эффективность работы и избежать перегрузок;

- Как руководитель проекта, я хочу получать автоматические отчеты о состоянии проекта и ключевых метриках через интеграцию с LLM-агентом, чтобы иметь актуальную информацию для презентаций и планирования.

б) Продукт-менеджер:

- Как продукт-менеджер, я хочу, чтобы LLM-ассистент мог анализировать бэклог задач и предлагать приоритизацию на основе бизнес-ценности и пользовательских требований, чтобы оптимизировать развитие продукта;

- Как продукт-менеджер, я хочу иметь возможность быстро создавать и обновлять пользовательские истории и требования через взаимодействие с LLM-агентом;

- Как продукт-менеджер, я хочу, чтобы LLM-ассистент мог автоматически формировать отчеты о выполнении эпиков и фич, чтобы отслеживать прогресс реализации стратегических целей продукта.

в) Разработчик:

[illegible]

- Как разработчик, я хочу, чтобы LLM-ассистент мог предоставлять контекст задачи из Яндекс Трекера непосредственно в моей IDE, чтобы быстрее понимать требования и начинать работу над задачей;
 - Как разработчик, я хочу, чтобы LLM-агент мог генерировать шаблоны кода и документации на основе спецификаций задач в Яндекс Трекере, чтобы ускорить процесс разработки;
 - Как разработчик, я хочу иметь возможность обновлять статус задачи и добавлять комментарии к задаче через LLM-ассистент, чтобы минимизировать переключение между инструментами и сосредоточиться на коде.
- г) Тестировщик:
- Как тестировщик, я хочу, чтобы LLM-ассистент мог автоматически создавать тест-кейсы на основе описания задачи в Яндекс Трекере, чтобы ускорить процесс тестирования и повысить его качество;
 - Как тестировщик, я хочу иметь возможность быстро добавлять баг-репорты в Яндекс Трекер через LLM-агента, интегрированный с моей IDE, чтобы сразу фиксировать найденные ошибки без переключения контекста;
 - Как тестировщик, я хочу, чтобы LLM-ассистент мог анализировать историю изменений задачи и предлагать сценарии регрессионного тестирования, чтобы обеспечить полноту покрытия тестами после изменений в коде.

1.2 Пользовательские сценарии

Пользовательские сценарии описывают типовые ситуации взаимодействия различных ролей с МСР-сервером для Яндекс Трекера, интегрированным с LLM-агентами. Они охватывают ключевые аспекты работы с системой управления задачами через интеллектуальных ассистентов.

а) Сценарий 1. Отслеживание прогресса проекта руководителем.

- Действующие лица: Руководитель проекта, LLM-ассистент, МСР-сервер, Яндекс Трекер;
- Цель: Получить актуальную информацию о состоянии проекта и ключевых метриках.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ Лист 7				
Изм.	Лист	№ докум.	Подп.	Дата					

- Руководитель проекта обращается к LLM-ассистенту с запросом "Покажи прогресс по проекту";
- LLM-ассистент отправляет запрос к MCP-серверу для получения данных из Яндекс Трекера;
- MCP-сервер извлекает информацию о задачах, статусах, сроках и загрузке команды;
- LLM-ассистент анализирует полученные данные и формирует отчет в понятной форме;
- Руководитель получает структурированную информацию о прогрессе проекта.

б) Сценарий 2. Приоритизация задач продукт-менеджером.

- Действующие лица: Продукт-менеджер, LLM-ассистент, MCP-сервер, Яндекс Трекер;
- Цель: Определить приоритеты задач на основе бизнес-ценности и пользовательских требований.
- Продукт-менеджер запрашивает у LLM-ассистента анализ бэклога задач;
- LLM-ассистент обращается к MCP-серверу за данными о задачах, их описании, метках и комментариях;
- MCP-сервер предоставляет структурированные данные из Яндекс Трекера;
- LLM-ассистент анализирует информацию и предлагает приоритезацию на основе бизнес-ценности;
- Продукт-менеджер получает рекомендации и может внести корректировки через LLM-ассистент.

в) Сценарий 3. Получение контекста задачи разработчиком.

- Действующие лица: Разработчик, LLM-ассистент, MCP-сервер, Яндекс Трекер;
- Цель: Быстро получить полный контекст задачи для начала работы.
- Разработчик начинает работу над задачей в своей IDE и запрашивает контекст;
- LLM-ассистент, интегрированный с IDE, обращается к MCP-серверу;

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Изм	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.842 ПЗ	Лист
											8

- МСР-сервер извлекает из Яндекс Трекера описание задачи, требования, комментарии и историю изменений;
 - LLM-ассистент обрабатывает информацию и предоставляет разработчику структурированный контекст;
 - Разработчик получает всю необходимую информацию для начала работы без переключения между инструментами.
- г) Сценарий 4. Создание баг-репорта тестировщиком.

- Действующие лица: Тестировщик, LLM-ассистент, МСР-сервер, Яндекс Трекер;
- Цель: Быстро зафиксировать найденную ошибку в системе управления задачами.
- Тестировщик находит баг и сообщает об этом LLM-ассистенту;
- LLM-ассистент собирает информацию о баге, включая шаги воспроизведения и ожидаемое поведение;
- LLM-ассистент отправляет запрос к МСР-серверу для создания задачи в Яндекс Трекере;
- МСР-сервер создает баг-репорт с заполненными полями на основе информации от LLM-ассистента;
- Тестировщик получает подтверждение создания баг-репорта и при необходимости может внести дополнения.

1.3 Основания для разработки

Основанием для разработки является задание на курсовой проект. Задание утверждено ООО «Философт», именуемый в дальнейшем Заказчиком, и Бикметовым Игорем Руслановичем (самозанятый), именуемым в дальнейшем исполнителем, Исполнитель обязан разработать «МСР-сервер для Яндекс Трекера» не позднее 26.12.2025. Наименование темы разработки – «Разработка МСР-сервера для Яндекс Трекера».

Изм.	Лист	№ докум.	Подп.	Дата
Инов. № подл.	Подп. и дата	Взам. инв. №	Инов. № дубл.	Подп. и дата

ТПЖА.090301.842 ПЗ

1.4 Назначение разработки

Данная система будет использоваться в рамках внутренней работы ООО «Философт» для интеграции Яндекс Трекера с LLM-агентами.

1.4.1 Функциональные требования

Ниже перечислены функциональные требования к МСР-серверу для Яндекс Трекера, сгруппированные по категориям:

1.4.1 Функциональные требования

Ниже перечислены функциональные требования к МСР-серверу для Яндекс Трекера, сгруппированные по категориям:

- а) Управление задачами:
 - Создание новых задач в системе Яндекс Трекер;
 - Чтение информации о задачах по их идентификаторам;
 - Обновление существующих задач с возвратом обновленной информации;
 - Удаление задач;
 - Поиск задач по различным критериям.
- б) Комментарии:
 - Создание комментариев к задачам;
 - Чтение комментариев задачи;
 - Обновление комментариев;
 - Удаление комментариев.
- в) Спринты:
 - Создание новых спринтов;
 - Чтение информации о спринтах;
 - Обновление информации о спринтах;
 - Удаление спринтов.
- г) Доски:
 - Создание новых досок;
 - Чтение информации о досках;
 - Обновление информации о досках;
 - Удаление досок.

д) Статусы:

- Создание новых статусов;
- Чтение информации о статусах;
- Обновление информации о статусах;
- Удаление статусов.

е) Очереди:

- Получение списка очередей задач;
- Получение детальной информации об очереди.

ж) Пользователи:

- Получение информации о текущем пользователе;
- Поиск пользователей по заданным критериям.

з) Ресурсы (resources):

- Предоставление информации о доступных типах задач в организации;
- Предоставление списка приоритетов задач;
- Предоставление списка доступных статусов задач.

и) Промпты (prompts):

- Анализ спринта или группы задач с предоставлением статистики и рекомендаций;
- Краткое изложение задачи с указанием её сути, статуса и исполнителей.

1.4.2 Эксплуатационное назначение

Система будет использоваться в рамках внутренней работы ООО «Философт» для оптимизации рабочего процесса.

1.5 Требования к МСР-серверу

В данном разделе будут описаны основные требования к МСР-серверу.

Инв. № подл.	Подп. и дата				Взам. инв. №	Инв. № дубл.				Подп. и дата			
– Краткое изложение задачи с указанием её сути, статуса и исполнителей.													
1.4.2 Эксплуатационное назначение													
Система будет использоваться в рамках внутренней работы ООО «Фило-софт» для оптимизации рабочего процесса.													
1.5 Требования к МСР-серверу													
В данном разделе будут описаны основные требования к МСР-серверу.													
					ТПЖА.090301.842 ПЗ								Лист
													11
Изм	Лист	№ докум.	Подп.	Дата									

1.5.1 Требования к составу выполняемых функций

а) Создание задачи (create_issue):

– Входные данные: заголовок (summary), описание (description), очередь (queue), тип (type), приоритет (priority), исполнитель (assignee), компоненты (components), метки (labels);

– Выходные данные: объект созданной задачи с уникальным ключом, содержащий все основные атрибуты задачи.

б) Чтение задачи (get_issue):

– Входные данные: идентификатор задачи (issueKey);

– Выходные данные: полная информация о задаче, включая статус, исполнителя, описание, комментарии, историю изменений.

в) Обновление задачи (update_issue):

– Входные данные: идентификатор задачи (issueKey), поля для обновления (summary, description, status, assignee и др.);

– Выходные данные: обновленный объект задачи с актуальной информацией.

г) Удаление задачи (delete_issue):

– Входные данные: идентификатор задачи (issueKey);

– Выходные данные: подтверждение успешного удаления задачи.

д) Поиск задач (search_issues):

– Входные данные: критерии поиска (язык запросов Трекера), параметры сортировки, лимит и смещение результатов;

– Выходные данные: список найденных задач с основной информацией.

е) Создание комментария (add_comment):

– Входные данные: идентификатор задачи (issueKey), текст комментария, упомянутые пользователи;

– Выходные данные: объект созданного комментария с идентификатором и содержимым.

ж) Чтение комментариев (get_comments):

– Входные данные: идентификатор задачи (issueKey), параметры пагинации;

Изм	Лист	№ докум.	Подп.	Дата	Инв. № инв.	Инв. № дубл.	Подп. и дата	Подп. и дата	Инв. № подл.	ТПЖА.090301.842 ПЗ		Лист
												12

– Выходные данные: список комментариев к задаче с информацией об авторе, дате и содержанием.

з) Обновление комментария (update_comment):

– Входные данные: идентификатор комментария, новое содержимое;

– Выходные данные: обновленный объект комментария.

и) Удаление комментария (delete_comment):

– Входные данные: идентификатор комментария;

– Выходные данные: подтверждение успешного удаления комментария.

к) Создание спринта (create_sprint):

– Входные данные: название спринта, дата начала, дата окончания, доска, к которой привязан спринт;

– Выходные данные: объект созданного спринта с идентификатором и основными атрибутами.

л) Чтение информации о спринте (get_sprint):

– Входные данные: идентификатор спринта;

– Выходные данные: полная информация о спринте, включая статус, даты, задачи, участников.

м) Обновление спринта (update_sprint):

– Входные данные: идентификатор спринта, поля для обновления (даты, статус, название);

– Выходные данные: обновленный объект спринта.

н) Удаление спринта (delete_sprint):

– Входные данные: идентификатор спринта;

– Выходные данные: подтверждение успешного удаления спринта.

о) Создание доски (create_board):

– Входные данные: название доски, тип доски, фильтр задач, настройки отображения;

– Выходные данные: объект созданной доски с идентификатором и конфигурацией.

п) Чтение информации о доске (get_board):

– Входные данные: идентификатор доски;

– Выходные данные: информация о доске, включая колонки, фильтры, настройки отображения.

р) Обновление доски (update_board):

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист				
										13				
										Изм	Лист	№ докум.	Подп.	Дата

- Входные данные: идентификатор доски, поля для обновления (название, фильтры, настройки);
- Выходные данные: обновленный объект доски.
- с) Удаление доски (delete_board):
 - Входные данные: идентификатор доски;
 - Выходные данные: подтверждение успешного удаления доски.
- т) Создание статуса (create_status):
 - Входные данные: название статуса, тип статуса (открыт, в работе, закрыт), цветовое обозначение;
 - Выходные данные: объект созданного статуса с идентификатором и атрибутами.
- у) Чтение информации о статусе (get_status):
 - Входные данные: идентификатор статуса;
 - Выходные данные: информация о статусе, включая его тип, цвет, описание.
- ф) Обновление статуса (update_status):
 - Входные данные: идентификатор статуса, поля для обновления (название, тип, цвет);
 - Выходные данные: обновленный объект статуса.
- х) Удаление статуса (delete_status):
 - Входные данные: идентификатор статуса;
 - Выходные данные: подтверждение успешного удаления статуса.
- ц) Получение очередей (get_queues):
 - Входные данные: параметры фильтрации и расширения;
 - Выходные данные: список очередей с информацией о каждой очереди.
- ч) Получение информации о очереди (get_queue):
 - Входные данные: идентификатор очереди, параметры расширения;
 - Выходные данные: детальная информация об очереди, включая настройки и права доступа.
- ш) Получение информации о пользователе (get_myself):
 - Входные данные: отсутствуют;
 - Выходные данные: информация о текущем пользователе, включая права доступа.
- щ) Поиск пользователей (search_users):

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист
										14
Изм.	Лист	№ докум.	Подп.	Дата						

- Входные данные: строка поиска, лимит результатов;
- Выходные данные: список найденных пользователей с основной информацией.

1.5.2 Требования к организации входных и выходных данных

Входные и выходные данные MCP-сервера должны соответствовать спецификации Model Context Protocol (MCP) версии 05.11.2024. Все данные передаются в формате JSON.

Входные данные:

- Все входящие запросы должны содержать корректный заголовок аутентификации (OAuth токен);
- Параметры запросов должны проходить валидацию с использованием схем, определенных в формате JSON Schema;
- Данные о задачах, пользователях, очередях и других сущностях должны соответствовать структуре, определенной в API Яндекс Трекера;
- Все строковые значения должны быть в кодировке UTF-8;
- Числовые значения должны соответствовать допустимым диапазонам, определенным в спецификации;
- Дата и время должны передаваться в формате ISO 8601.

Выходные данные:

- Все ответы сервера должны содержать корректный HTTP-статус в зависимости от результата операции;
- В случае ошибки, сервер должен возвращать детализированное сообщение об ошибке в стандартном формате MCP;
- Все объекты задач, пользователей, очередей и других сущностей должны содержать полный набор атрибутов в соответствии с API Яндекс Трекера;
- Объем возвращаемых данных должен быть ограничен параметрами пагинации, если это применимо;
- Все данные должны быть защищены от несанкционированного доступа, токены и конфиденциальная информация не должны включаться в ответы сервера.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	– Все строковые значения должны быть в кодировке UTF-8; – Числовые значения должны соответствовать допустимым диапазонам, определенным в спецификации; – Дата и время должны передаваться в формате ISO 8601.	
					Выходные данные:	
					– Все ответы сервера должны содержать корректный HTTP-статус в зависимости от результата операции; – В случае ошибки, сервер должен возвращать детализированное сообщение об ошибке в стандартном формате MCR; – Все объекты задач, пользователей, очередей и других сущностей должны содержать полный набор атрибутов в соответствии с API Яндекс Трекера; – Объем возвращаемых данных должен быть ограничен параметрами пагинации, если это применимо; – Все данные должны быть защищены от несанкционированного доступа, токены и конфиденциальная информация не должны включаться в ответы сервера.	
Изм	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.842 ПЗ	Лист
						15

1.5.3 Требования к временным характеристикам

Максимальное время отклика системы 1 секунда.

1.5.4 Требования к надежности

Вероятность безотказной работы сервера должна составлять не менее 90.99% при условии исправности соединения с Яндекс Трекером.

1.5.5 Требования к обеспечению надежного функционирования системы

Для обеспечения надежного функционирования системы необходимо выполнение следующих требований:

а) Система должна быть спроектирована с учетом отказоустойчивости, чтобы минимизировать влияние возможных сбоев компонентов на общую работоспособность системы;

б) Система должна поддерживать механизм мониторинга состояния компонентов и логирования событий для своевременного выявления и устранения неполадок;

в) Должна быть обеспечена защита от несанкционированного доступа к данным и функциям системы с использованием современных методов аутентификации и авторизации;

г) Система должна поддерживать масштабирование для обеспечения стабильной работы при увеличении нагрузки.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист				
										16				
										Изм	Лист	№ докум.	Подп.	Дата

1.5.6 Время восстановления после отказа

Время восстановления после отказа, вызванного неисправностью технических средств, не должно превышать времени, требуемого на устранение неисправностей технических средств.

1.5.7 Условия эксплуатации

Для обеспечения надежной и безопасной эксплуатации МСР-сервера необходимо соблюдение следующих условий:

- а) Сервер должен эксплуатироваться в закрытых помещениях с контролируемым климатом, без прямого воздействия солнечных лучей и атмосферных осадков;
- б) Температурный режим эксплуатации: от $+10^{\circ}\text{C}$ до $+35^{\circ}\text{C}$;
- в) Напряжение питания: $220\text{ В} \pm 10\%$, частота $50\text{ Гц} \pm 2\%$;
- г) Сервер должен быть защищен от пыли, вибраций и механических воздействий;
- д) Запрещена эксплуатация в условиях агрессивной среды (наличие химически активных газов, паров кислот и щелочей);
- е) Обслуживание МСР-сервера должно производиться квалифицированным персоналом, прошедшим инструктаж по технике безопасности;
- ж) Сервер должен быть подключен к стабилизированному источнику питания с возможностью резервного питания на время кратковременных отключений;
- з) Для МСР-сервера должна быть обеспечена надежная связь с Яндекс Трекером через защищенный канал связи;
- и) Сервер должен находиться в контролируемом (дата-центре или серверной комнате) с ограниченным физическим доступом.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист
										17
Изм.	Лист	№ докум.	Подп.	Дата						

1.5.8 Климатические условия эксплуатации

Специальные условия не требуются.

1.5.9 Требования к видам обслуживания

Обслуживание МСР-сервера включает в себя следующие виды работ:

а) Техническое обслуживание:

- Регулярный контроль работоспособности системы (ежедневно);
- Проверка состояния логов и мониторинг производительности (ежедневно);
- Обновление программного обеспечения (по мере необходимости);
- Проверка целостности данных и резервных копий (еженедельно).

б) Профилактическое обслуживание:

- Проверка и тестирование резервного копирования данных (ежемесячно);
- Анализ производительности и выявление узких мест (ежемесячно);
- Проверка конфигурации безопасности и настройка доступа (ежеквартально);
- Проверка и обновление документации по системе (ежеквартально).

в) Ремонтное обслуживание:

- Устранение неисправностей в работе системы (по мере возникновения);
- Восстановление работоспособности после сбоев (при необходимости);
- Замена или восстановление компонентов системы (при необходимости);
- Восстановление из резервной копии в случае потери данных (при необходимости).

Все виды обслуживания должны выполняться квалифицированным персоналом с соблюдением установленных процедур и регламентов безопасности.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата						Лист
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.842 ПЗ					18

1.5.10 Требования к численности и квалификации персонала

Обслуживание МСР-сервера должно осуществляться специалистом с высшим техническим образованием и опытом работы в области разработки и сопровождения серверных приложений не менее 2 лет. Для обеспечения бесперебойной работы системы рекомендуется наличие дежурного системного администратора с соответствующей квалификацией.

1.5.11 Требование к маркировке и упаковке

Специальных требований к маркировке и упаковке не предъявляется.

1.5.12 Требования к транспортированию и хранению

Специальных требований не предъявляется.

1.5.13 Специальные требования

Специальных требований не предъявляется.

1.6 Требования к программной документации

Предварительный состав программной документации:

- а) техническое задание (включает описание применения);
- б) методика испытаний;
- в) документация по МСР-серверу для Яндекс Трекера.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист
										19
Изм.	Лист	№ докум.	Подп.	Дата						

1.7 Технико-экономические показатели

МСР-сервер будет использоваться в рамках внутренней работы ООО «Философт». Это позволит оптимизировать рабочий процесс персонала. Что, в свою очередь, сократит время на реализацию поставленных задач.

1.8 Стадии и этапы разработки

Разработка должна быть проведена в три стадии:

- а) техническое задание;
- б) технический (и рабочий) проекты;
- в) внедрение.

На стадии «Техническое задание» должен быть выполнен этап разработки, согласования и утверждения настоящего технического задания. На стадии «Технический (и рабочий) проект» должны быть выполнены перечисленные ниже этапы работ:

- а) разработка сервера;
- б) разработка документации;
- в) испытания сервера.

На стадии «Внедрение» должен быть выполнен этап разработки «Подготовка и передача сервера». Содержание работ по этапам: На этапе разработки технического задания должны быть выполнены перечисленные ниже работы:

- а) постановка задачи;
- б) определение и уточнение требований к техническим средствам;
- в) определение требований к серверу;
- г) определение стадий, этапов и сроков разработки сервера и документации на него;
- д) согласование и утверждение технического задания.

Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.842 ПЗ	Лист		
							20	

1.9 Порядок контроля и приемки

Приемосдаточные испытания МСР-сервера должны проводиться согласно разработанной исполнителем и согласованной заказчиком «Методики испытаний». Ход проведения приемо-сдаточных испытаний заказчик и исполнитель документируют в протоколе испытаний. На основании протокола испытаний исполнитель совместно с заказчиком подписывают акт приемки-сдачи МСР-сервера в эксплуатацию.

1.10 Заключение

Настоящее техническое задание определяет требования к разработке МСР-сервера для Яндекс Трекера, предназначенного для интеграции с LLM-агентами. Документ охватывает все аспекты разработки, включая функциональные и нефункциональные требования, условия эксплуатации, требования к персоналу и документации. Выполнение всех указанных в техническом задании требований обеспечит создание надежного и эффективного решения для автоматизации работы с Яндекс Трекером. Техническое задание является основанием для выполнения работ по разработке и последующему внедрению МСР-сервера в ООО «Философт».

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист
										21
Изм.	Лист	№ докум.	Подп.	Дата						

Копировал

Формат А4

2 Анализ предметной области

2.1 Общая характеристика системы

МСР-сервер для Яндекс Трекера представляет собой специализированное программное решение, предназначенное для обеспечения интеграции между системой управления задачами Яндекс Трекер и интеллектуальными агентами на основе Large Language Models (LLM). Основной целью системы является предоставление LLM-агентам доступа к информации о проекте через протокол Model Context Protocol (MCP), что позволяет автоматизировать взаимодействие с задачами, улучшить контекстное понимание проектных сущностей и оптимизировать рабочие процессы в командной разработке.

Система функционирует как посредник между Яндекс Трекером и LLM-ассистентами, обеспечивая выполнение широкого спектра операций: создание, чтение, обновление и удаление задач, комментариев, спринтов, досок и других элементов проекта. Также сервер предоставляет информацию о пользователях, очередях задач, статусах и других ресурсах, необходимых для полноценной интеграции. Важной особенностью является поддержка промптинга - возможность анализа данных проекта и генерации рекомендаций на основе искусственного интеллекта.

Сервер должен соответствовать требованиям производительности (максимальное время отклика 1 секунда) и надежности (вероятность безотказной работы не менее 90.99%), а также обеспечивать безопасное взаимодействие с внешней системой через аутентификацию с использованием OAuth токенов. Архитектура сервера должна быть спроектирована с учетом отказоустойчивости и возможности масштабирования при увеличении нагрузки.

2.2 Анализ существующих проблем

В современных условиях разработка программного обеспечения требует слаженного взаимодействия между различными ролями в команде: руководите-

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата						
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.842 ПЗ					Лист
										22

лями, продукт-менеджерами, разработчиками и тестировщиками. В организации применяется Яндекс Трекер для управления проектами и часто возникают следующие проблемы:

а) Фрагментация информации: Команда разработки сталкивается с необходимостью переключения между различными инструментами (IDE, система управления задачами, чаты и др.) для выполнения повседневных задач. Это приводит к снижению продуктивности и увеличению времени на выполнение рутинных операций;

б) Недостаточная автоматизация: Рутинные операции, такие как создание задач, обновление статусов, добавление комментариев, требуют ручного вмешательства и отнимают значительное количество времени у специалистов;

в) Сложности в анализе данных: Руководителям и продукт-менеджерам сложно оперативно получать аналитическую информацию о состоянии проекта, загрузке команды и приоритетности задач без ручного анализа данных в Яндекс Трекере;

г) Отсутствие интеллектуальной поддержки: Отсутствует интеграция с интеллектуальными агентами, которые могли бы автоматизировать анализ задач, генерировать отчеты, предлагать оптимизацию распределения задач и создавать тест-кейсы;

д) Задержки в коммуникации: Коммуникация между членами команды может замедляться из-за необходимости переключения между инструментами для получения контекста задачи, особенно для разработчиков, которым нужно понимать требования и спецификации задачи;

е) Недостаточная интеграция с ИИ: Отсутствует прямая интеграция Яндекс Трекера с LLM-агентами, что не позволяет использовать возможности искусственного интеллекта для автоматизации процессов, анализа данных и принятия решений.

Данные проблемы особенно актуальны для команд, которые используют Яндекс Трекер как основной инструмент управления проектами, но не имеют полноценной интеграции с современными ИИ-ассистентами. Решение этих проблем возможно через разработку MCP-сервера, который обеспечит прямое взаимодействие между Яндекс Трекером и LLM-агентами, позволяя автоматизировать рутинные задачи и улучшить общую эффективность работы команды.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист
										23
Изм	Лист	№ докум.	Подп.	Дата						

Таблица 1 - Анализ существующих проблем

Название проблемы	Краткое описание
Фрагментация информации	Переключение между инструментами снижает продуктивность
Недостаточная автоматизация	Рутинные операции требуют ручного вмешательства
Сложности в анализе данных	Трудности с получением аналитической информации
Отсутствие интеллектуальной поддержки	Нет интеграции с ИИ-агентами
Задержки в коммуникации	Затруднено получение контекста задачи
Недостаточная интеграция с ИИ	Отсутствует прямая интеграция с LLM-агентами

2.3 Анализ требований по решению существующих проблем

С точки зрения системного анализа и заказчика, МСР-сервер должен решать выявленные проблемы за счет реализации следующих требований и функциональных возможностей:

- а) Устранение фрагментации информации:
 - Обеспечение прямой интеграции с IDE и инструментами разработки через MCP-протокол, что позволяет получать информацию из Яндекс Трекера без переключения между приложениями;
 - Предоставление контекста задачи непосредственно в рабочем окружении разработчика;
 - Генерация краткого изложения задачи с указанием её сути, статуса исполнителей;
 - Обогащение контекста LLM-модели под реализацию конкретной задачи.
- б) Повышение уровня автоматизации:
 - Реализация полного цикла операций с задачами, комментариями, спринтами и досками;

Подп. и дата		С точки зрения системного анализа и заказчика, МСР-сервер должен решать выявленные проблемы за счет реализации следующих требований и функциональных возможностей: а) Устранение фрагментации информации: – Обеспечение прямой интеграции с IDE и инструментами разработки через МСР-протокол, что позволяет получать информацию из Яндекс Трекера без переключения между приложениями; – Предоставление контекста задачи непосредственно в рабочем окружении разработчика; – Генерация краткого изложения задачи с указанием её сути, статуса исполнителей; – Обогащение контекста LLM-модели под реализацию конкретной задачи. б) Повышение уровня автоматизации: – Реализация полного цикла операций с задачами, комментариями, спринтами и досками;					
Инв. № дубл.							
Взам. инв. №							
Подп. и дата							
Инв. № подл.							
Изм	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.842 ПЗ		Лист
							24

– Автоматизация создания и обновления задач через интеграцию с LLM-ассистентами, что исключает необходимость ручного ввода данных в интерфейсе Яндекс Трекера;

– Поддержка поиска задач с различными критериями для автоматического анализа и обработки задач.

в) Упрощение анализа данных:

– Делегирование анализа данных LLM-модели;

– Предоставление информации о пользователях и очередях для анализа загрузки команды;

– Анализ спринта или группы задач с предоставлением статистики и рекомендаций;

– Получение детальной информации о задачах, включая историю изменений и комментарии, для автоматического формирования отчетов.

г) Обеспечение интеллектуальной поддержки:

– Интеграция с LLM-агентами через MCP-протокол для автоматического анализа задач и генерации рекомендаций;

– Генерация тест-кейсов на основе описания задачи через интеллектуальный анализ содержимого задач;

– Приоритезация задач на основе анализа бизнес-ценности и пользовательских требований.

д) Ускорение коммуникации:

– Обеспечение мгновенного доступа к контексту задачи из IDE или других инструментов разработки без необходимости переключения на интерфейс Яндекс Трекера;

– Быстрое добавление комментариев и обновления статусов задач непосредственно из рабочего окружения разработчика;

– Интеграция системами чатов и коммуникации для автоматического уведомления участников проекта о изменениях.

е) Интеграция с ИИ-технологиями:

– Обеспечение соответствия спецификации Model Context Protocol для полноценной интеграции с LLM-агентами;

– Предоставление информации о типах задач, приоритетах и статусах в формате, понятном для ИИ-ассистентов;

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

Изм	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.842 ПЗ

- Реализация безопасного и надежного взаимодействия с Яндекс Трекером через аутентификацию и защиту от несанкционированного доступа.

Таким образом, МСР-сервер решает существующие проблемы за счет создания единой точки интеграции между Яндекс Трекером и интеллектуальными агентами, обеспечивая автоматизацию рутинных операций, улучшение контекста для разработчиков и аналитиков, а также повышение общей эффективности командной работы. Выполнение всех указанных требований позволит значительно сократить время на реализацию проектов и повысить качество взаимодействия между членами команды.

2.4 Обзор существующих решений

В рамках анализа предметной области и постановки задачи разработки МСР-сервера для Яндекс Трекера, было выявлено несколько существующих аналогичных решений, которые обеспечивают интеграцию Яндекс Трекера с интеллектуальными агентами через МСР-протокол. Ниже представлены основные аналоги и их характеристики:

а) Официальный MCP-сервер от Yandex Cloud:Официальный шаблон MCP-сервера для Yandex Tracker, представленный в Yandex AI Studio;

Преимущества:

- Официальная поддержка от Яндекса;
- Интеграция с Yandex AI Studio;
- Гарантированная совместимость с API Яндекс Трекера.

Недостатки:

- Ограниченный функционал по сравнению с требованиями проекта;
- Ограниченная кастомизация под специфические требования;
- Возможная зависимость от инфраструктуры Яндекса;
- Меньшая гибкость в настройке под внутренние процессы организации.

б) Открытые (OpenSource) решения: Различные реализации МСР-серверов для Яндекс Трекера на репозиториях GitHub и NPM. Анализ данных проектов выявил преимущества и недостатки, которые описаны далее.

Преимущества:

Изм.	Лист	№ докум.	Подп.	Дата	логи и их характеристики: а) Официальный MCP-сервер от Yandex Cloud:Официальный шаблон MCP-сервера для Yandex Tracker, представленный в Yandex AI Studio; Преимущества: – Официальная поддержка от Яндекса; – Интеграция с Yandex AI Studio; – Гарантированная совместимость с API Яндекс Трекера. Недостатки: – Ограниченный функционал по сравнению с требованиями проекта; – Ограниченная кастомизация под специфические требования; – Возможная зависимость от инфраструктуры Яндекса; – Меньшая гибкость в настройке под внутренние процессы организации. б) Открытые (OpenSource) решения: Различные реализации MCP-серверов для Яндекс Трекера на репозиториях GitHub и NPM. Анализ данных проектов выявил преимущества и недостатки, которые описаны далее. Преимущества:

- Открытый исходный код, позволяющий адаптацию под конкретные нужды;
- Возможность модификации и расширения функциональности;
- Независимость от конкретного поставщика;
- Сообщество разработчиков, способствующее улучшению решений.

Недостатки:

- Открытый исходный код может облегчить злоумышленникам поиск уязвимостей;
- Отсутствие централизованной поддержки и гарантий.

2.5 Сравнительный анализ решений

Для принятия обоснованного решения при выборе подхода к разработке МСР-сервера для Яндекс Трекера, проведем сравнительный анализ существующих решений по следующим критериям:

а) Полный набор функций — наличие всех необходимых функций, соответствующих требованиям технического задания, включая создание, чтение, обновление и удаление задач, комментариев, спринтов, досок и других сущностей;

б) Безопасность использования — соблюдение принципов информационной безопасности;

в) Бесплатное решение — отсутствие платы за использование решения;

г) Возможность масштабирования — способность системы адаптироваться к увеличению нагрузки, количества пользователей и объема обрабатываемых данных без значительного снижения производительности, а также возможность расширения функционала.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист	
Изм.	Лист	№ докум.	Подп.	Дата						27	

Таблица 2 - Сравнительный анализ решений

Решение	Полный набор функций	Безопасность использования	Бесплатное решение	Возможность масштабирования
Официальный МСР-сервер от Yandex Cloud	Нет	Да	Нет	Нет
OpenSource решения	Да	Нет	Да	Да
Разрабатываемое решение	Да	Да	Да	Да

Таким образом, анализ существующих аналогов показывает, что, несмотря на наличие официального решения от Яндекса и open-source альтернатив, разрабатываемое решение имеет существенные преимущества для внутреннего использования в ООО «Философт», обусловленные спецификой требований заказчика, возможностью кастомизации и учетом внутренних процессов разработки.

2.6 Выбор архитектуры

В данном разделе представлено сравнение различных архитектурных подходов и обоснован выбор архитектуры для реализации проекта. Для начала рассмотрим преимущества и недостатки:

а) Многослойная архитектура

Преимущества:

- Простая реализация
- Разделение ответственностей между уровнями
- Изолирование защищает одни слои от изменений других
- Повышение управляемости за счет слабой связанности

Недостатки:

- Слабая масштабируемость
- Монолитная структура, усложняющая модификацию

Инов. № подл.	Подп. и дата	Взам. инв. №	Инов. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист
										28
Изм.	Лист	№ докум.	Подп.	Дата						

– Данные должны проходить по каждому слою, даже если нет необходимости передавать их с определенных слоев

б) Многоуровневая архитектура

Преимущества:

- Простота развертывания
- Минимальные затраты на инфраструктуру
- Легкость в отладке и тестировании

Недостатки:

- Ограниченное масштабирование
- Сложность обновления - высокая зависимость между компонентами

в) Сервис-ориентированная архитектура

Преимущества:

- Слабая связанность
- Гибкость
- Возможность повторного использования

Недостатки:

- Сложность
- Сетевая зависимость

г) Микросервисная архитектура

Преимущества:

- Слабая связанность благодаря высокой степени изоляции
- Модульность
- Сбой в одном сервисе не затронет всю систему, поскольку они изолиро-

ваны

- Высокая гибкость и масштабируемость
- Простота модификации может ускорить итерации

Недостатки:

- Повышенный риск сбоя при обмене данными между сервисами
- Большим количеством сервисов трудно управлять
- Задержки в сети, необходимость балансировки нагрузки
- Нуждается в комплексном тестировании в распределенной среде

д) Бессерверная архитектура

Преимущества:

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ	Лист				
						29				
						Изм.	Лист	№ докум.	Подп.	Дата

– Удобство - разработчики получают простую в развертывании и использовании среду, в которой автоматизированы практически все технологические процессы

– Производительность - в бессерверных вычислениях разработчики легко получают необходимую мощность процессора, объем памяти, место в хранилище и другие ресурсы, необходимые для выполнения конкретных запросов

– Гибкость - управление бессерверными вычислениями лежит полностью на провайдере, он предоставляет вычислительные мощности автоматически, без прямого участия разработчиков

– Простое масштабирование - в бессерверных вычислениях в конкретный момент времени все исполняемые фрагменты кода получают необходимые для работы ресурсы

– Экономичность - разработчики оплачивают только фактически используемые ресурсы, избегая переплат при непостоянной нагрузке

– Стабильность - вычислительные ресурсы доступны всегда из любой точки мира, доступность, безопасность и отказоустойчивость IT-инфраструктуры полностью обеспечивает провайдер

Недостатки:

– Низкий уровень управляемости

– Не поддерживают все языки программирования (но поддерживают все самые популярные), не являются унифицированными (у каждого поставщика услуги - своя специфика), наблюдается задержки при запуске «холодных» функций

Для более детального сравнения архитектур, были определены следующие характеристики оценки:

– Эффективность системы - способность архитектуры обеспечивать высокую производительность и минимальные задержки при обработке запросов;

– Гибкость системы - возможность адаптации архитектуры к изменяющимся требованиям и условиям эксплуатации;

– Расширяемость системы - возможность добавления новых функций и компонентов без значительных изменений в существующую структуру;

– Масштабируемость процесса разработки - возможность эффективного распределения задач между разработчиками и командами;

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата						
Изм	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.842 ПЗ					Лист
										30

- г) Производительность: микросервисная архитектура позволяет оптимизировать каждый компонент отдельно для обеспечения высокой производительности;
- д) Возможность независимой разработки и деплоя: различные компоненты сервера могут разрабатываться и обновляться независимо;
- е) Соответствие современным тенденциям: микросервисная архитектура соответствует современным тенденциям разработки серверных приложений, особенно для систем, требующих интеграции с ИИ-агентами и высокой доступности.

2.7 Заключение

В рамках данного раздела была проведена комплексная оценка текущего состояния интеграции Яндекс Трекера с интеллектуальными агентами. Были выявлены ключевые проблемы, с которыми сталкиваются участники команды разработки: фрагментация информации, недостаточная автоматизация процессов, сложности в анализе данных, отсутствие интеллектуальной поддержки, задержки в коммуникации и недостаточная интеграция с ИИ-технологиями.

Анализ существующих решений показал, что официальный MCP-сервер от Yandex Cloud имеет ограничения по функциональности и гибкости, в то время как open-source решения, хотя и обеспечивают возможность кастомизации, не гарантируют уровня безопасности, необходимого для внутреннего использования в организации.

На основе проведенного анализа было принято решение о разработке собственного MCP-сервера, который объединит преимущества существующих решений и будет адаптирован под специфические требования ООО «Философт». Выбор микросервисной архитектуры обоснован необходимостью обеспечения высокой масштабируемости, гибкости, отказоустойчивости и возможности независимой разработки компонентов. Это позволит создать надежное и эффективное решение, соответствующее современным требованиям к интеграции с LLM-агентами и обеспечивающее автоматизацию рабочих процессов в командной разработке.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ки в коммуникации и недостаточная интеграция с ИИ-технологиями.	
					Анализ существующих решений показал, что официальный МСР-сервер от Yandex Cloud имеет ограничения по функциональности и гибкости, в то время как open-source решения, хотя и обеспечивают возможность кастомизации, не гарантируют уровня безопасности, необходимого для внутреннего использования в организации.	
					На основе проведенного анализа было принято решение о разработке собственного МСР-сервера, который объединит преимущества существующих решений и будет адаптирован под специфические требования ООО «Философт».	
					Выбор микросервисной архитектуры обоснован необходимостью обеспечения высокой масштабируемости, гибкости, отказоустойчивости и возможности независимой разработки компонентов. Это позволит создать надежное и эффективное решение, соответствующее современным требованиям к интеграции с LLM-агентами и обеспечивающее автоматизацию рабочих процессов в командной разработке.	
					ТПЖА.090301.842 ПЗ	Лист
						32
Изм	Лист	№ докум.	Подп.	Дата		

3 Выбор технологий и инструментов для реализации проекта

В рамках разработки МСР-сервера для Яндекс Трекера необходимо провести отбор и обоснование технологий, языков программирования, фреймворков, библиотек инструментов, которые будут использоваться в проекте. Правильный выбор технологий играет ключевую роль в обеспечении надежности, производительности, безопасности и поддерживаемости конечного решения. В данном разделе будут рассмотрены и обоснованы все ключевые аспекты технологического стека, используемого для реализации МСР-сервера.

3.1 Анализ и обоснование выбора языка программирования

При выборе языка программирования для реализации МСР-сервера были рассмотрены следующие языки: TypeScript, JavaScript, Python, Java и Go. Ниже приведен анализ преимуществ и недостатков каждого из них:

а) TypeScript:

Преимущества:

- Статическая типизация, позволяющая выявлять ошибки на этапе компиляции;
- Совместимость с JavaScript и возможность использования существующих библиотек;
- Поддержка современных стандартов и улучшенная поддержка IDE;
- Отличная интеграция с Node.js.

Недостатки:

- Необходимость компиляции перед выполнением.

б) PHP:

Преимущества:

- Простота и быстрая разработка веб-приложений;
- Широкая поддержка и обширная экосистема библиотек;
- Хорошая интеграция с веб-серверами и базами данных.

Недостатки:

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата						Лист
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.842 ПЗ					33

– Меньшая производительность по сравнению с компилируемыми языками;

- Сложности с масштабированием и поддержкой крупных проектов;
- Отсутствие строгой типизации (до версии 7.0).

в) Python:

Преимущества:

- Простота и читаемость кода;
- Богатая стандартная библиотека и поддержка веб-фреймворков;
- Отличная поддержка для работы с API и сетевыми запросами.

Недостатки:

– Меньшая производительность по сравнению с компилируемыми языками;

- Проблемы с параллелизмом из-за GIL (Global Interpreter Lock).

г) Java:

Преимущества:

- Высокая производительность и надежность;
- Статическая типизация и строгая проверка типов;
- Отличная поддержка многопоточности и масштабируемых приложений.

Недостатки:

– Более громоздкий синтаксис и необходимость написания большего объема кода;

- Более длительный цикл разработки и компиляции.

д) Go:

Преимущества:

- Высокая производительность и быстрая компиляция;
- Простота синтаксиса и встроенная поддержка конкурентности;
- Хорошие возможности для создания сетевых сервисов.

Недостатки:

- Меньшая экосистема библиотек по сравнению с TypeScript или Python.

На основе проведенного анализа был выбран TypeScript как оптимальный язык программирования для реализации МСР-сервера. Основные причины выбора:

- Развитая инфраструктура для реализации МСР-серверов;

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист
										34
Изм.	Лист	№ докум.	Подп.	Дата						

- Совместимость с JavaScript и экосистемой Node.js позволяет использовать готовые библиотеки для HTTP-запросов, работы с JSON и асинхронными операциями, что критически важно при интеграции с API Яндекс Трекера;
- Статическая типизация позволяет определять структуры данных, получаемых от API Яндекс Трекера, обеспечивая корректную работу с полями задач, комментариев, спринтов и других сущностей, а также предотвращая ошибки при обращении к ним.

3.2 Выбор библиотек

Для реализации MCP-сервера для Яндекс Трекера были выбраны следующие библиотеки:

- @modelcontextprotocol/sdk - официальный SDK для создания MCP-серверов, предоставляет готовые классы и интерфейсы для реализации протокола Model Context Protocol, включая сервер, транспортные стратегии и вспомогательные утилиты;
- @modelcontextprotocol/inspector - инструмент для отладки и тестирования MCP-серверов, используется для проверки корректности реализации протокола;
- Express - библиотека для создания веб-сервера и маршрутизации HTTP-запросов, используется для обработки SSE-соединений и других HTTP-взаимодействий;
- yandex-tracker-client - библиотека для работы с API Яндекс Трекера, предоставляет готовые методы для выполнения операций с задачами, комментариями, спринтами и другими сущностями;
- Zod - библиотека для валидации данных, используется для проверки входных параметров и обеспечения соответствия структур данных требованиям API;
- Pino - библиотека для логирования событий в приложении, обеспечивает структурированное и производительное логирование;
- dotenv - библиотека для загрузки переменных окружения из .env файлов, используется для безопасного хранения конфигурационных параметров.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист	
Изм.	Лист	№ докум.	Подп.	Дата						35	

Выбор этих библиотек обусловлен необходимостью обеспечить надежное взаимодействие с API Яндекс Трекера, поддержку MCR-протокола, высокую производительность, безопасность и удобство разработки.

3.3 Выбор системы контроля версий

Рассмотрим две основные системы контроля версий: Git и Subversion (SVN). Ниже приведен анализ каждой из них.

а) Git

Преимущества:

- Распределенность: каждый разработчик имеет полную копию репозитория с историей изменений, что обеспечивает автономную работу и защиту от потери данных при сбоях центрального сервера;
- Производительность: локальные операции выполняются быстро, так как большая часть данных хранится локально;
- Гибкое ветвление и слияние: мощные инструменты для работы с ветками, позволяющие параллельно работать над различными функциями;
- Интеграция с платформами: хорошая интеграция с GitHub, GitLab, Bitbucket;
- Поддержка CI/CD: интеграция с системами непрерывной интеграции и доставки;
- Прозрачность и аудит: полная история изменений с информацией о внешних изменениях;
- Стандарт в индустрии: широкое использование в сообществе разработчиков.

Недостатки:

- Отсутствие детального контроля доступа: Git не позволяет ограничивать доступ к конкретным частям репозитория, предоставляя пользователям только полный доступ или доступ только для чтения;
- Необходимость резервного копирования: Git не предоставляет встроенных механизмов автоматического резервного копирования, поэтому командам

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ми, позволяющие параллельно работать над различными функциями;				
					– Интеграция с платформами: хорошая интеграция с GitHub, GitLab, Bitbucket;				
					– Поддержка CI/CD: интеграция с системами непрерывной интеграции и доставки;				
					– Прозрачность и аудит: полная история изменений с информацией о внесенных изменениях;				
					– Стандарт в индустрии: широкое использование в сообществе разработчиков.				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Недостатки:				
					– Отсутствие детального контроля доступа: Git не позволяет ограничивать доступ к конкретным частям репозитория, предоставляя пользователям только полный доступ или доступ только для чтения;				
					– Необходимость резервного копирования: Git не предоставляет встроенных механизмов автоматического резервного копирования, поэтому командам				
					ТПЖА.090301.842 ПЗ				
36									
Изм	Лист	№ докум.	Подп.	Дата					

необходимо самостоятельно обеспечивать создание резервных копий репозитория.

б) Subversion (SVN)

Преимущества:

- Централизованная модель: проще в понимании для новичков, так как работает с центральным репозиторием;
- Права доступа: гибкая настройка прав доступа к отдельным папкам репозитория;
- Работа с большими файлами: более эффективно работает с большими файлами и бинарными данными.

Недостатки:

- Необходимость подключения к серверу: для большинства операций требуется подключение к центральному серверу;
- Меньшая гибкость ветвления: ветвление и слияние менее гибкие по сравнению с Git;
- Меньше инструментов CI/CD: меньше систем непрерывной интеграции.

Для более обоснованного выбора системы контроля версий были определены следующие критерии оценки:

а) Производительность: скорость выполнения основных операций (клонирование, коммит, пуш, пул, ветвление, слияние) как при локальной работе, так и при взаимодействии с удаленным репозиторием. Этот критерий важен для обеспечения высокой скорости разработки и сокращения времени ожидания при выполнении операций с репозиторием;

б) Надежность: способность системы сохранять целостность данных, обеспечивать восстановление после сбоев, а также защищать от потери информации при различных аварийных ситуациях. Этот критерий важен для обеспечения стабильности разработки и предотвращения потери результатов работы команды;

в) Уровень интеграции с CI/CD инструментами: наличие готовых решений и поддержка со стороны систем непрерывной интеграции и доставки, таких как Jenkins, GitLab CI, GitHub Actions и других. Этот критерий важен для автоматизации процессов тестирования, сборки и деплоя, что позволяет ускорить разработку и повысить качество конечного продукта.

Таблица 4 - Сравнение систем контроля версий по ключевым критериям

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист	
Изм.	Лист	№ докум.	Подп.	Дата						37	

3.4 Заключение

В рамках реализации проекта по созданию МСР-сервера для Яндекс Трекера был проведен анализ. Основным языком программирования выбран TypeScript, который обеспечивает статическую типизацию, совместимость с JavaScript-экосистемой и улучшенную поддержку IDE, что особенно важно при работе с API Яндекс Трекера.

Для реализации МСР-протокола и взаимодействия с API Яндекс Трекера были выбраны специализированные библиотеки, включая официальный SDK для МСР, Express для веб-сервера, yandex-tracker-client для работы с API Яндекс Трекера, Zod для валидации данных и Pino для логирования.

В качестве системы контроля версий выбран Git, который обеспечивает распределенность, высокую производительность и отличную интеграцию с современными CI/CD-инструментами. Для хранения репозитория используется GitHub, что обеспечивает надежное хранение кода и инструменты для совместной работы.

Выбранный технологический стек позволяет создать надежное, производительное и масштабируемое решение, соответствующее современным требованиям к интеграции с LLM-агентами.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.842 ПЗ					Лист
										39
Изм.	Лист	№ докум.	Подп.	Дата						

4 Проектирование

Инв. № подл.		Подп. и дата		Взам. инв. №		Инв. № дубл.		Подп. и дата		ТПЖА.090301.842 ПЗ					Лист
															40
Изм.	Лист	№ докум.	Подп.	Дата											